Design Principles & Patterns

Java Backend Interview — Short Notes

1. SOLID Principles

Principle	Definition	Violated When	Realistic Java Example
S — Single Responsibility	A class should have only one reason to change.	UserService sends email, validates, persists, generates reports — 4 reasons to change.	Split into UserService, UserValidator, EmailSender, UserReportService.
O — Open/Closed	Open for extension, closed for modification.	Adding new payment type requires editing PaymentProcessor switch statement.	PaymentProcessor interface. Add StripeProcessor implements PaymentProcessor — no existing code changes.
L — Liskov Substitution	Subtypes must be usable wherever base type is expected.	Square extends Rectangle. setWidth(5) on Square also changes height — breaks rectangle contract.	Don't force IS-A relationships. Use composition or separate interface.
I — Interface Segregation	Clients should not depend on interfaces they don't use.	Animal interface with fly(), swim(), run() — Dog forced to implement fly().	Flyable, Swimmable, Runnable interfaces. Dog implements Swimmable, Runnable only.
D — Dependency Inversion	Depend on abstractions, not concretions.	OrderService directly instantiates MySQLOrderRepository — hard to test, hard to swap.	OrderService depends on OrderRepository interface. Spring injects MySQLOrderRepository. Swap to MongoOrderRepository in tests.

2. Other Key Design Principles

Principle	Meaning	Example
DRY — Don't Repeat Yourself	Every piece of knowledge should have a single representation. Avoid duplication.	Extract common validation logic to a Validator class instead of copying in every service.
KISS — Keep It Simple	Prefer simple solutions. Complexity = bugs + maintenance burden.	Don't design for hypothetical future requirements. Solve today's problem simply.
YAGNI — You Aren't Gonna Need It	Don't add features/abstractions until actually needed.	Don't add a plugin system 'just in case'. Build when the requirement exists.
Composition over Inheritance	Favour composing objects over extending classes.	Duck has a FlyBehavior field (composition) vs Duck extends FlyingAnimal (inheritance). Behaviour changeable at runtime.
Law of Demeter	Talk only to immediate friends. Don't chain: a.getB().getC().doWork().	order.getCustomer().getAddress().getCity() — violates. Pass city directly or add order.getCustomerCity().
Fail Fast	Detect and report errors as early as possible.	Validate inputs at controller layer. Throw immediately on null/invalid — don't pass bad data downstream.
Encapsulate What Varies	Identify aspects that change and separate from what stays the same.	Tax calculation changes per country — encapsulate in TaxStrategy. Checkout flow stays stable.

3. Creational Design Patterns

Pattern	Intent	Realistic Java Use Case	Key Code Structure
Singleton	Ensure only one instance of a class exists globally.	Spring beans are singletons by default. Logger, ConfigManager, ConnectionPool.	Private constructor + static getInstance(). Thread-safe: enum or double-checked locking.
Factory Method	Define interface for creating object, let subclasses decide which class to instantiate.	NotificationFactory.create('EMAIL') returns EmailNotification. Decouples client from concrete class.	Creator abstract class with createProduct(). Subclasses override.
Abstract Factory	Create families of related objects without specifying concrete classes.	UIFactory: WindowsUIFactory or MacUIFactory. Returns matching Button+Checkbox+TextField for platform.	Factory interface with multiple create methods. Concrete factory per product family.
Builder	Construct complex objects step by step.	HttpRequest.newBuilder().uri(...).header(...).POST(body).build(). Avoids telescoping constructors.	Builder inner class. Fluent setters returning this. build() validates and creates.
Prototype	Clone existing object instead of creating from scratch.	Clone a template document configuration. Cache frequently used complex objects and clone per request.	Implement Cloneable + clone(). Or copy constructor. Deep vs shallow clone.

```
// Builder pattern — realistic example
OrderRequest request = OrderRequest.builder()
.productId("prod-123")
.quantity(2)
.shippingAddress(address)
.couponCode("SAVE10")
.build(); // validates all required fields in build()
// Factory method — notification routing
public interface Notification { void send(String message); }
public class NotificationFactory {
public static Notification create(String channel) {
return switch (channel) {
case "EMAIL" -> new EmailNotification();
case "SMS" -> new SmsNotification();
case "PUSH" -> new PushNotification();
default -> throw new IllegalArgumentException("Unknown: " + channel);
};
}
}
```

4. Structural Design Patterns

Pattern	Intent	Realistic Java Use Case
Adapter	Convert interface of a class into another interface clients expect.	Legacy PaymentGateway has charge(int cents). New system needs PaymentPort.pay(BigDecimal). Adapter wraps legacy class.
Decorator	Add responsibilities to object dynamically without subclassing.	Java I/O: new BufferedInputStream(new FileInputStream(...)). Add compression, encryption, logging to base stream.
Proxy	Control access to another object (lazy init, caching, logging, security).	Spring AOP @Transactional/@Cacheable creates proxy. Hibernate lazy loading = proxy. Spring Security method security = proxy.

Facade	Provide simplified interface to complex subsystem.	OrderFacade.placeOrder() internally calls InventoryService, PaymentService, ShippingService, EmailService — client sees one method.
Composite	Treat individual objects and compositions uniformly (tree structure).	File system: File and Directory both implement FileSystemComponent. calculate size() works on both leaves and branches.
Bridge	Decouple abstraction from implementation so both can vary independently.	Shape (abstraction) has Renderer (implementation). Circle+Square work with SVGRenderer+OpenGLRenderer independently.
Flyweight	Share common state among many fine-grained objects.	String pool in Java. Character objects in text editors. Tile objects in game maps — share type data, vary position.

```
// Decorator pattern — realistic: adding logging to a service
public class LoggingOrderService implements OrderService {
private final OrderService delegate;
private final Logger log = LoggerFactory.getLogger(getClass());
public Order createOrder(OrderRequest req) {
log.info("Creating order for product: {}", req.getProductId());
Order result = delegate.createOrder(req);
log.info("Order created: {}", result.getId());
return result;
}
}
```

5. Behavioural Design Patterns

Pattern	Intent	Realistic Java Use Case
Strategy	Define family of algorithms, encapsulate each, make them interchangeable.	SortStrategy: BubbleSort, QuickSort, MergeSort. PaymentStrategy: CreditCard, PayPal, Crypto. Switch at runtime.
Observer	One-to-many dependency: when one object changes, all dependents notified.	Spring ApplicationEventPublisher. Kafka producer/consumer. GUI button listeners. Stock price → multiple dashboards.
Command	Encapsulate a request as an object. Support undo, queuing, logging.	Undo/redo in text editors. Job queue: each job is a Command. Spring @Scheduled tasks. Transaction scripts.
Template Method	Define skeleton of algorithm in base class, defer steps to subclasses.	Spring JdbcTemplate: execute(). AbstractDataMigration: validate(), transform(), load() — subclass overrides transform().
Chain of Responsibility	Pass request through chain of handlers. Each decides to handle or pass on.	Spring Security FilterChain. Servlet filters. Logging levels. Exception handler chain.
State	Object changes behavior when internal state changes.	Order: PENDING→PAID→SHIPPED→DELIVERED. Traffic light. VendingMachine idle/selecting/dispensing states.
Iterator	Traverse collection without exposing implementation.	Java Iterator/Iterable. Enhanced for-loop. Stream pipeline. Database cursor.
Mediator	Object that encapsulates how other objects interact.	Chat room mediator. Spring ApplicationContext as bean mediator. API Gateway routing. MVC Controller.

```
// Strategy pattern — payment processing
public interface PaymentStrategy {
PaymentResult pay(BigDecimal amount);
```

```
}
// Checkout service uses strategy — no switch/if chains
public class CheckoutService {
private final Map<String, PaymentStrategy> strategies;
public CheckoutService(List<PaymentStrategy> strategies) {
this.strategies = strategies.stream()
.collect(toMap(s -> s.getType(), identity()));
}
public void checkout(Cart cart, String paymentType) {
strategies.get(paymentType).pay(cart.getTotal());
}
}
// Observer pattern — Spring ApplicationEvent
// Publisher:
applicationEventPublisher.publishEvent(new OrderCreatedEvent(order));
// Subscriber 1: send email
@EventListener public void onOrder(OrderCreatedEvent e) { emailSvc.send(e.getOrder()); }
// Subscriber 2: update analytics
@EventListener public void trackOrder(OrderCreatedEvent e) { analytics.track(e); }
```

6. Interview Questions

SOLID

Q1. Explain Open/Closed Principle with a real example.

→ PaymentProcessor with switch(paymentType) violates OCP — adding PayPal requires modifying the class. Fix: PaymentStrategy interface with CreditCardStrategy, PayPalStrategy, CryptoStrategy implementations. PaymentProcessor depends on interface — extension by adding a new class, no modification.

Q2. How does Spring Boot demonstrate Dependency Inversion?

→ Spring IoC container manages beans. Classes depend on interfaces (OrderRepository), not implementations (JpaOrderRepository). Spring injects the concrete implementation. In tests, inject mock implementations without changing OrderService. This is DIP in practice.

Q3. What violates Liskov Substitution Principle?

→ Classic: Square extends Rectangle. setWidth(5) also changes height (to maintain square invariant), breaking Rectangle behavior. Code expecting Rectangle breaks when given Square. Fix: don't inherit. Use separate Quadrilateral interface or composition. Liskov: any property provable of Rectangle must also hold for Square — it doesn't.

Patterns

Q4. When would you use Decorator vs Inheritance?

→ Decorator adds behavior at runtime to individual instances without affecting others. Multiple decorators can be stacked. Inheritance adds behavior to all instances of a subclass, compile-time only. Use decorator when: behaviors need to be combined flexibly (logging + caching + retry on a service), or when subclassing would cause class explosion (LoggingCachingRetryingOrderService).

Q5. What pattern does Spring AOP implement?

→ Proxy pattern. Spring wraps beans with dynamic proxies (JDK or CGLIB). @Transactional, @Cacheable, @Async, @PreAuthorize — all implemented as proxy interceptors that add cross-cutting behavior transparently. The bean doesn't know it's being proxied.

Q6. Singleton in Spring — is it thread-safe?

→ Spring manages singleton lifecycle — one instance shared across all threads. The bean itself must be stateless (no mutable instance fields) to be thread-safe. If a singleton has state, use ThreadLocal, synchronize, or change scope to @RequestScope. Spring's singleton scope = one instance per ApplicationContext (not JVM).

Q7. What is the difference between Factory Method and Abstract Factory?

→ Factory Method: one factory method, one product type. Subclasses decide which concrete product to create. Abstract Factory: creates families of related products. All created products are designed to work together. Factory Method is about one product, Abstract Factory is about product families (e.g., UI components for a platform).

Q8. How does the Observer pattern differ from Pub/Sub?

→ Observer: direct coupling — subject holds references to observers and calls them directly. Synchronous. Pub/Sub: decoupled via message broker (Kafka, RabbitMQ) — publisher and subscriber don't know each other. Asynchronous. Spring ApplicationEvent is observer. Kafka is pub/sub. Pub/Sub scales better, observer is simpler.